

10. Develop an express web application that can interact with REST API to perform CRUD operations on student data. (Use Postman)

app.js

```
const express = require('express');
const app = express();
app.use(express.json());
```

```
let students = [];
```

```
// CREATE
```

```
app.post('/add', (req,res)=>{
  students.push(req.body);
  res.json({msg:"Added", data: students});
});
```

```
// READ
```

```
app.get('/all', (req,res)=>{
  res.json(students);
});
```

```
// UPDATE
```

```
app.put('/update/:id', (req,res)=>{
  students[req.params.id] = req.body;
  res.json({msg:"Updated", data: students});
});
```

```
// DELETE
```

```
app.delete('/delete/:id', (req,res)=>{
  students.splice(req.params.id,1);
  res.json({msg:"Deleted", data: students});
});
```

```
app.listen(3000, ()=>console.log("Server running on 3000"));
```

DELETE localhost/delete/1

Params Authorization Headers (8) **Body** Pre-request Script Tests Settings

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL [JSON](#)

```
1 {
2   "id": 1
3 }
4
```

Body Cookies Headers (7) Test Results 200 OK

Pretty Raw Preview Visualize JSON

```
1 {
2   "message": "Deleted,data:student"
3 }
```

POST /add

```
{
  "name": "Ram"
}
```

Response:

```
{
  "msg": "Added",
  "data": [
    { "name": "Ram" }
  ]
}
```

GET /all

Response:

```
[
  { "name": "Ram" }
]
```

PUT /update/0

```
{
  "name": "Sam"
}
```

Response:

```
{  
  "msg": "Updated",  
  "data": [  
    { "name": "Sam" }  
  ]  
}
```

DELETE /delete/1**Response:**

```
{  
  "msg": "Deleted",  
  "data": []  
}
```